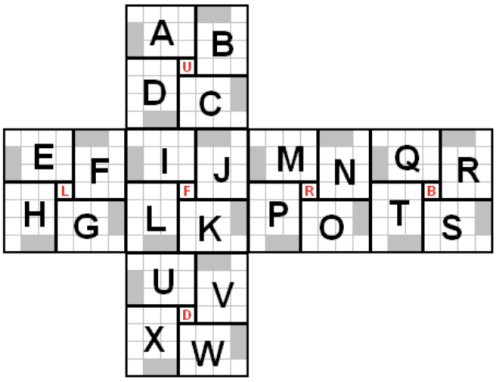
Solving a 2x2 Rubik's Cube Using a LEGO EV3 Robot

Kosta Gjorgjievski, Srinjana Sriram, Harshith Senthilkumaran

1 Problem Statement

This project aims to develop a LEGO robot capable of solving a 2x2 Rubik's Cube. The robot must:

- Perceive the initial cube state using sensors
- Compute a sequence of moves to solve the cube
- Physically manipulate the cube using a robotic gripper or arm to execute the required rotations



Speffz Letter Scheme

2 System Design and Methodology

2.1 Hardware Setup

We used the LEGO Mindstorms EV3 kit as the core hardware platform, utilizing its built-in sensors, motors, and joints. The EV3 brick runs MicroPython via the Pybricks library, enabling low-level control of the robot. Programs were uploaded to the EV3 brick via a laptop.

Listing 1: Hardware Initialization

```
from pybricks.ev3devices import Motor,
    ColorSensor
from pybricks.parameters import Port
motor = Motor(Port.A)
sensor = ColorSensor(Port.S3)
```

2.2 Cube Scanning

The cube is scanned by rotating it and reading each face with the color sensor. Each color is stored in an array using the Speffz letter scheme to represent the cube's state.

Listing 2: Cube Flipping Example

```
def flip():
    arm.run_until_stalled(-200, then=Stop.
        BRAKE, duty_limit=None)
    wait(500)
    ev3.speaker.beep()
    # Additional rotation logic
```

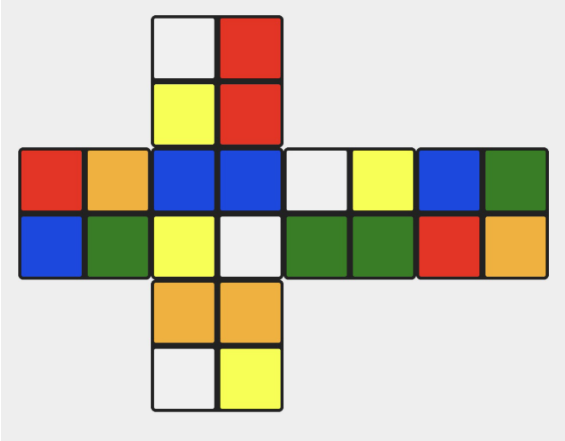
2.3 Cubie Color to State Conversion and Encoding

To solve the cube algorithmically, the robot must interpret scanned colors into a format a solver can understand. Each corner cubie on a 2x2 Rubik's Cube is uniquely defined by its color combination. Our system converts the scanned 24-color array into a cube state consisting of two parts:

- **Permutation (8 values):** Identifies which cubie is in each of the 8 positions. For example, if the array begins with [5, 3, 7...], it means that cubie #5 is currently in position 0, cubie #3 is in position 1, and so on.
- **Orientation (8 values):** Each cubie has 3 possible twists depending on which of its stickers is facing up/down. These are encoded as integers

0, 1, or 2. The orientation helps determine if a cubie is rotated correctly in its position.

These two arrays are concatenated into a single 16-element state. To optimize lookup time in our solution table, one cubie (the DBL piece) is fixed, reducing the state to 14 numbers. This compressed state is then encoded into a single integer used for fast table lookup.



Cubie State Representation with Colors

2.4 Table Lookup

We used a precomputed binary file `massive_table.bin` generated using Eric Wu's *Massive Table Generator* [1]. Each unique cube state maps to a human-readable solution sequence.

Listing 3: Binary Lookup Access

```
offset = state_integer * MOVE_SIZE
file.seek(offset)
moves = file.read(MOVE_SIZE)
```

2.5 Human Solution Moves to Robot Actions

Human moves were translated into robot motor commands. These instructions were then executed by EV3 motors.

3 Evaluation Results

We validated components such as motor control and color detection. The final robot was tested on five scrambled cubes, solving four successfully.

Demo: Video Demonstration (YouTube)

4 Discussion and Reflections

4.1 Hardware Issues

The EV3 kit was sourced second-hand and lacked parts, so we modified David Gilday's MindCub3r design [2] to fit a 2x2 cube, resizing components like the turntable and flipper arm.

4.2 Sensor Challenges

The EV3 sensor had trouble differentiating orange and yellow, so we painted one face black and used matte tape to reduce reflections.

4.3 Optimizations for Table Access

To manage the large binary file (22MB), we encoded cube states as integers and used `seek()` to access specific bytes without loading the entire file.

Team Contribution Division

- **Kosta Gjorgjievski:** Led the complete robot setup, constructed joints and mounted sensors, handled arm calibration, and integrated hardware with software using Pybricks. Also helped with debugging sensor detection.
- **Srinjana Sriram:** Collaborated on writing the logic to interpret human-readable solutions from the lookup table and converting them into robot instructions. Worked closely with solver pipeline and validation.
- **Harshith Senthilkumaran:** Focused on refining robotic motion, calibrating rotation angles for smoother manipulation, and testing precise execution of commands to ensure high-speed and accurate solutions.

Code Explanation and Structure

main.py

This script handles the full robotic operation. It initializes and calibrates motors (for flipping and rotating the cube), the color sensor, and its mechanical arm. The cube scanning procedure involves scanning one

face at a time, flipping or rotating as needed, and mapping readings into a 24-element array using the Speffz scheme. The scanned colors are converted into a 16-integer state and passed to the solver. The final move sequence is executed by translating abstract moves into motor operations like `flip()`, `rotate_left()`, and `turn_right()`.

cube.py

This module defines how cube states are represented and manipulated. A state consists of 8 cubie positions and 8 orientations. It includes functions to rotate the cube, apply moves, and convert 16-state representations into a compact 14-state form, fixing one cubie for optimization. It also implements low-level move effects and validates physical plausibility of scanned states.

solver.py

This module performs table-based solving. It converts the 14-state into a unique integer index, retrieves a 6-byte solution from `massive_table.bin`, and decodes it into human-readable moves. It then translates these moves into robot-compatible moves (`D`, `D'`, `D2`, `z'`, `y`, etc.) using rotation logic. It encapsulates the high-level intelligence of the robot by bridging abstract solutions with physical execution.

Code Repository

The full project codebase, including all scripts, configurations, and instructions, is available at:
`https://github.com/srinji5141/2x2RubiksCubeRobot/tree/srinjana`

The project website can be found here:
`https://srinji5141.github.io/2x2RubiksCubeRobot`

References

[1] Eric Wu, *Massive Table Generator for Rubik's Cube*, 2024. GitHub: `https://github.com/ericwu17/Massive-Table-Generator/`

[2] David Gilday, *MindCub3r for LEGO EV3*, `https://mindcuber.com/mindcub3r/mindcub3r.html`